LAPORAN PRAKTIKUM DEVOPS & CLOUD COMPUTING

BAB 7: MONITORING RESOURCE DENGAN PROMETHEUS, CADVISOR, NODE EXPORTOR, DAN GRAFANA

IDENTITAS MAHASISWA

- **Nama Lengkap** : Mohammad Affan Hendi Firmansyah
- **NIM** : 3126640049
- **Kelas** : 1 IT-B S.Tr.Lj

RINGKASAN EKSEKUTIF

Praktikum Bab 7 berfokus pada perancangan, implementasi, dan pengujian arsitektur *Monitoring Resource* pada lingkungan kontainer Docker menggunakan stack standar industri: **Prometheus** sebagai time-series database dan scraper, **cAdvisor** untuk metrik container, **Node Exporter** untuk metrik host sistem, serta **Grafana** sebagai dashboard dan visualisasi.

Topik utama meliputi: perbedaan antara monitoring metrics dan logging events, model *pull* scrape Prometheus, pengumpulan time-series data, dan integrasi data log dari praktikum Bab 6 ke dalam Grafana untuk korelasi insiden lintas observability stack.

Kompetensi utama yang dicapai meliputi:

1. Menjalankan Prometheus sebagai scraper dan time-series database
2. Mengumpulkan metrik host dengan Node Exporter dan metrik container dengan cAdvisor
3. Membuat dashboard Grafana untuk CPU, memory, disk, network, dan container usage
4. Mengintegrasikan data log PostgreSQL dari Bab 6 ke Grafana

Seluruh target pengujian berhasil divalidasi dengan status **PASS**:

1. Semua service Prometheus stack (`prometheus`, `grafana`, `node-exporter`, `cadvisor`) berstatus `Up` dan terdeteksi oleh `docker compose ps`.
2. Endpoint Prometheus `http://127.0.0.1:9090` merespons dengan query up menampilkan 5 target status teratas.
3. Dashboard Grafana berhasil dimuat dan menampilkan panels untuk CPU usage, memory consumption, dan container status.
4. Data source PostgreSQL Bab 6 dapat ditambahkan ke Grafana dengan hak baca-only (`grafana_reader`) dan sukses menjalankan query sederhana terhadap tabel `logs.fluentbit`.
5. Konfigurasi recording rules dan alert rules Prometheus dapat diaktifkan tanpa error saat `promtool check rules`.

BAB VII: KONSPIL DAN LANDASAN TEORI

BAB VII: LANGKAH-LANGKAH PRAKTIKUM

7.1 Penyiapan Lingkungan

```
mkdir -p ~/docker-lab/bab-7/{prometheus/rules,prometheus/tests,grafana/provisioning/datasources,grafana/provisioning/dashboards,grafana/dashboards,scripts,reports}
cd ~/docker-lab/bab-7
```

7.2 Konfigurasi `.env` dan `.gitignore`

```
cp .envexample .env
chmod 600 .env
```

Credential tersebut bersifat sintetis. Pada lingkungan produksi, mengganti dengan Docker Swarm Secrets atau HashiCorp Vault.

7.3 Menjalankan Stack Monitoring

```
docker compose up -d
```

Service yang akan dijalankan: `prometheus`, `grafana`, `node-exporter`, `cadvisor`, dan `generator` (beban resource).

7.4 Verifikasi Stack

```
docker compose past
```

Pastikan semua target Prometheus berstatus up.

7.5 Menambahkan Data Source PostgreSQL Bab 6

Jalankan skrip integrasi:

```
bash scripts/connect-bab6.sh
```

Skrip ini melakukan koneksi ke database PostgreSQL dari Bab 6 dengan hak hanya baca (`SELECT`) pada skema `logs`, menggunakan kredensial dari berkas `.env`.

7.6 Akses Antarmuka

- **Prometheus**: `http://127.0.0.1:9090`
- **Grafana**: `http://127.0.0.1:3000` (user: admin, password: grafanaadmin123)

7.7 Bukti Eksekusi (Screenshot Evidence)

Bukti Minimum Bab 7:

1. Screenshot `docker compose ps` yang menunjukkan service berstatus `Up` (prometheus, grafana, node-exporter, cadvisor).
2. Screenshot hasil akses `http://localhost:9090` dan query PromQL up, `rate(container_cpu_usage_seconds_total[5m])`, `container_memory_usage_bytes`, `node_filesystem_avail_bytes`.
3. Screenshot dashboard Grafana dengan panels CPU, memory, dan container metrics.
4. Screenshot hasil koneksi data source PostgreSQL Bab 6 di Grafana (settings > data sources > PostgreSQL Bab 6 test connection sukses).
5. Screenshot query sukses terhadap `logs.fluentbit` dari within Grafana PostgreSQL data source (misal: `SELECT count(*) FROM logs.fluentbit GROUP BY tag`).

7.8 Troubleshooting dan Analisis Hasil

Gejala	Penyebab yang Mungkin	Tindakan Korektif
Gate berbeda antara lokal dan pipeline	Versi tool, input efektif, atau konfigurasi tidak sama	Pin versi; simpan konfigurasi efektif dan identitas artefak.
Service sehat tetapi security gate gagal	Healthcheck hanya memeriksa availability	Tinjau policy, scan, identity, signature, dan evidence secara terpisah.
Evidence tidak dapat ditelusuri	Commit, digest, waktu, atau owner tidak dicatat	Gunakan manifest evidence dan metadata yang konsisten.
Deployment gagal dipulihkan	Rollback, backup, atau credential rotation belum diuji	Lakukan recovery exercise dan dokumentasikan hasilnya.

7.9 Evaluasi dan Latihan Mandiri

1. Mengapa Prometheus menggunakan scrape model dan bukan hanya push model?
2. Apa dampak scrape_interval terlalu kecil terhadap storage dan network?
3. Mengapa `rate` diperlukan untuk counter seperti `node_cpu_seconds_total`?
4. Apa indikator awal container mengalami memory pressure?
5. Bagaimana menghubungkan alert metrics dengan pencarian log?
6. Hitung estimasi penggunaan CPU host tanpa recording rule, lalu bandingkan hasil dan biaya query dengan recording rule.
7. Ubah interval scrape menjadi 5 detik dan 30 detik. Ukur dampak terhadap jumlah sampel serta kemampilan melihat spike singkat.
8. Identifikasi label cAdvisor dengan cardinality tertinggi. Jelaskan label mana yang layak dipertahankan atau dipindahkan ke log.
9. Tambahkan panel network receive/transmit menggunakan `rate(node_network_receive_bytes_total[5m])` dan `rate(node_network_transmit_bytes_total[5m])`.
10. Hubungkan kenaikan CPU container dengan log Bab 6 pada rentang waktu yang sama dan jelaskan apakah korelasi tersebut cukup untuk menyatakan sebab-akibat.
11. Rancang SLI dan SLO sederhana untuk service aplikasi; jelaskan mengapa CPU tidak dapat menjadi pengganti langsung untuk availability pengguna.

7.10 Integrasi Bab 6 → Bab 7: PostgreSQL ke Grafana

Skenario integrasi mencakup:

1. Menambahkan PostgreSQL sebagai data source tambahan di Grafana dengan koneksi `postgres-db:5432`.
2. Menggunakan kredensial `grafana_reader`/`grafanareaderpass123` dengan hak `SELECT` terbatas pada skema `logs`.
3. Menjalankan query berikut di Grafana PostgreSQL data source untuk menampilkan distribusi log per generator:

```
SELECT
    tag,
  count(*) AS total_logs
 FROM logs.fluentbit
    GROUP BY tag
ORDER BY total_logs DESC;
```

1. Membuat panel Grafana yang menampilkan correlasi antara kenaikan penggunaan CPU (dari `container_cpu_usage_seconds_total`) dengan peningkatan volume log (dari `logs.fluentbit` count) pada rentang waktu yang sama.

7.11 Referensi Utama

1. Prometheus — Configuration.
2. Prometheus — Query functions.
3. Grafana — Provisioning.
4. Grafana — Data sources.
5. Prometheus Node Exporter releases.
6. Google cAdvisor releases.

7.1 Monitoring vs Logging

Monitoring menjawab pertanyaan "apa yang sedang terjadi pada sistem" melalui angka time-series. Logging menjawab "mengapa hal itu terjadi" melalui event dan pesan detail. Keduanya harus digunakan bersama agar incident response tidak berhenti pada gejala.

7.2 Model Pull vs Push pada Prometheus

Prometheus memakai model *pull*: server melakukan scrape endpoint `/metrics` dari target secara berkala. Grafana tidak menyimpan metrics utama, tetapi meng-query data source seperti Prometheus atau PostgreSQL untuk visualisasi dan alerting.

7.3 PromQL dan Agregasi Temporal

PromQL memungkinkan agregasi temporal seperti `rate`, `avg`, `sum`, dan histogram. Kesalahan umum pada pemula adalah membaca counter mentah tanpa `rate`, sehingga grafik terlihat naik terus tanpa merepresentasikan throughput aktual.

7.4 Komponen Arsitektur

Komponen	Fungsi	Port umum
Prometheus	Scrape metrics dan menyimpan time-series	9090
Node Exporter	Expose metrics host: CPU, RAM, disk, network	9100
cAdvisor	Expose metrics container Docker	8080/8081
Grafana	Dashboard dan visualisasi data source	3000
PostgreSQL datasource	Menampilkan log query dari Bab 6	5432

7.5 Setiap time series dibentuk oleh nama metric dan kombinasi label

Label seperti `service`, `instance`, `method`, `status`, dan `environment` membantu segmentasi. Namun, memasukkan user ID, request ID, alamat acak, atau pesan error sebagai label menciptakan cardinality tinggi yang memperburuk penggunaan memori dan storage.